

interpreter handled correctly, if it is run from a shell (both were limitations in the early Bourne shell's script handling), shell scripts are set up and executed by the OS itself. Nowadays, a modern shell script is not just on the same footing as the system commands, but rather, many of the system commands are actually shell scripts. This extends to the returning exit codes like other system utilities in order to indicate success or failure, and also allows them to be called components of the larger programs, regardless of how all those larger tools are implemented. Like different standard system commands, shell scripts omit any kind of file name extension unless it is intended to be read into a running shell using a special mechanism for this purpose (such as `csh`'s `source` or `sh`'s `'.'`).

Shell scripting on various operating systems

It is possible to leverage the various advantages of shell scripting on different operating systems (other than UNIX and Windows) as well with the help of interoperability software. Let's have a look at some of these options.

1. Interoperability software like Cygwin, Interix (which is available in the Microsoft Windows Services for UNIX), MKS Toolkit, UWIN (AT&T UNIX for Windows), Hamilton C shell, and others, allow UNIX shell programs to be run on different Windows NT machines and all their successors, with some loss of functionality on the MS-DOS-Windows 95 branch, as well as earlier MKS Toolkit versions for OS/2. At least three DCL implementations for Windows OS – apart from XLNT, a multiple-use scripting language package (which can be used with the command shell), CGI programming and Windows Script Host -- are available for these types of systems as well.
2. In addition to all these tools, some OS/2 and Posix functionality can also be used with the corresponding environmental sub-systems of the Windows NT OS series up to Windows 2000 as well. A third 16-bit sub-system, often called the MS-DOS sub-system, uses *Command.com* which is provided with these OSs to run all the just-mentioned MS-DOS batch files.
3. Different console alternatives such as 4DOS, 4NT, 4OS2, Peter Norton's NDOS, FreeDOS, and the GUI *Take Command*, which add the functionality to Windows NT-style *Cmd.exe*, OS/2's *Cmd.exe*, MS-DOS/Windows 95 batch files (run by *Command.com*), and 4NT, respectively, are similar to the shells that they actually enhance and are much more integrated with the Windows script host. The Windows Script Host comes with three engines (VBScript, VBA and JScript) that are pre-installed, and to which a number of third-party engines can be added with Perl, Rexx, Python, Tcl and Ruby with pre-defined functions in 4NT. PC DOS is also quite similar to MS-DOS, while DR DOS is quite

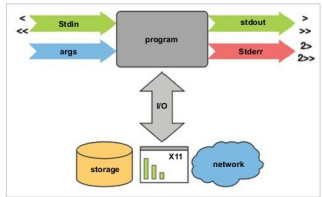


Figure 4: Output redirection and piping in shell scripting
(Image credits: googleimages.com)

different. Various earlier versions of Windows NT are able to successfully run contemporary versions of 4OS2 with the help of the OS/2 sub-system.

4. Scripting languages can be extended; for example, Windows NT and MS-DOS/Windows 95/98 type systems allowed for batch/shell programs to call different tools like KixTart, QBasic, Rexx; various Basic, Python and Perl implementations; and the Windows Script Host and all its installed engines. On UNIX and other Posix-compliant systems, Sed and Awk are used to extend the string and the numeric processing ability of shell scripts. Tcl, Rexx, Perl and Python have graphics toolkits that can be used to code different procedures and functions for shell scripts. This poses a speed bottleneck (Fortran, C, and Assembly language are much faster still) and adds functionality such as sockets and other connectivity functions, which are not available in the shell language. VBA and VBScript can be easily used to communicate with databases, spreadsheets, scriptable program of all types, development tools, telecommunications software, graphics tools and other software that can be accessed using the Component Object Model.

Writing your first shell script

Well, we are now at the most interesting part, which is, programming using the shell script. So we will actually follow three simple steps in order to successfully write our first shell script, which is the famous program to display 'Hello World'. The pre-requisites are:

- You need to have Linux installed on your system.
- You need to have a text editor (select from Vim, kwrite, etc), as well, on your system.

Now, let's follow the three simple steps.

1. Write the script

A shell script is basically a file that contains ASCII text. In order to create a shell script, we use a text editor, which is nothing but a program, similar to a word processor, which reads and writes different ASCII text files. There are many text editors available for our Linux system, both for the GUI